Improving the Tactics `ring` and `field`

Thomas Portet

October 16, 2026

Abstract

This document describes an extension of Rocq's [**?**] reflexive algebraic tactics. The main goal is to make `ring` and `field` usable on expressions that contain functions outside the ring or field fragment. The approach rewrites the arguments of such functions, computes polynomial greatest common divisors with an OCaml oracle, and exposes the result to Rocq through ELPI foreign calls.

1 Motivation

We would like to formalize undergraduate mathematics in a style that remains close to ordinary pen-and-paper reasoning, while retaining the formal guarantees of a proof assistant. We therefore choose to work in a simple ambient theory, with a small collection of familiar mathematical objects and operations, and to allow users to express mathematical arguments through high-level declarative steps rather than low-level proof-assistant transformations.

This setting motivates the question of how algebraic automation should behave when an algebraic expression occurs inside a function that is not itself part of the algebraic theory handled by the tactic. For example, a student should ideally be able to write

$$\log(\pi^2 + \pi \times \pi) = \log(2\pi^2)$$

without having to explicitly expose the intermediate algebraic rewriting

$$\pi^2 + \pi \times \pi = 2\pi^2.$$

The proof assistant should be able to recognize and justify this transformation automatically.
The difficulty is that the outer function ln is not a ring operation. Consequently, the algebraic tactic cannot treat the whole expression as a ring expression.

The existing `ring_simplify` tactic addresses this issue for ring expressions by recursively simplifying algebraic expressions inside function applications. However, the analogous approach does not directly extend to field expressions. This raises the question of how to obtain a suitable representation of field expressions when they occur inside functions that the field normalizer does not handle.

2 Reflexive Algebraic Normalization

The starting point is the way Rocq's reflexive algebraic tactics normally solve equalities. Rather than applying a sequence of rewriting lemmas, the tactic reifies the input expression into an internal representation of the relevant algebraic structure, computes its normal form, and uses this

computation to construct a proof. This approach is exemplified by the reflective ring tactic of Grégoire and Mahboubi [**?**].

2.1 Ring normalization

The `ring` tactic works with integer constants, addition, multiplication, subtraction, and variables. Expressions outside this language are treated as fresh variables during reification. Thus, for an expression such as

$$\pi^2 + \pi \times \pi,$$

the constant π can be represented by a fresh variable X. The tactic then normalizes the resulting polynomial expression. When the normalized expression is reconstructed as a Rocq term, the fresh variable X is replaced by the original expression π.

More generally, the process can be described in three steps:

1. reify the input expression, replacing supported operations by constructors in an internal polynomial representation and treating unsupported expressions as fresh variables;
2. normalize the resulting polynomial by collecting coefficients and combining like monomials;
3. conclude when the normalized expressions are syntactically equal.

Although the arguments of the two logarithms are polynomial expressions, the complete terms

$$\log(\pi^2 + \pi \times \pi) \qquad \text{and} \qquad \log(2\pi^2)$$

are themselves outside the polynomial language. At the level at which `ring` sees them, they therefore become distinct atoms. The tactic cannot discover that these atoms are equal by simplifying their arguments.

2.2 Deep ring simplification

The natural way to address this limitation is to apply ring simplification recursively inside function applications. Instead of treating an entire unrecognized function application as an indivisible atom, we first simplify the expressions occurring in its arguments and then reconsider the enclosing expression.

For example,

$$\cos(x + \cos(x + x)) = \cos(\cos(2x) + (1 - (1 - x))).$$

The relevant subexpressions include

$$x + \cos(x + x), \qquad x + x, \qquad \cos(2x) + (1 - (1 - x)), \qquad \text{and} \qquad 2x.$$

The inner expression $x + x$ can first be normalized to $2x$. The enclosing expression can then be reified again, with $\cos(2x)$ as an atom, allowing

$$\cos(2x) + (1 - (1 - x))$$

to be normalized to

$$x + \cos(2x).$$

The original equality consequently reduces to

$$\cos(x + \cos(2x)) = \cos(x + \cos(2x)).$$

This gives a natural recursive strategy for expressions containing unrecognized functions: simplify their arguments, rebuild the function applications from the simplified arguments, and continue at enclosing levels.

The same idea applies to the motivating logarithm example. The argument

$$\pi^2 + \pi \times \pi$$

can be normalized to $2\pi^2$ before the surrounding ln application is considered. The two sides then contain the same function application, so the outer equality can be closed.

This approach works naturally for `ring` because polynomial normalization provides a canonical representation of the algebraic expression. For `field`, however, the situation is more subtle.

2.3 Ring simplification versus field simplification

The recursive strategy described above is implemented by `ring_simplify`, which simplifies algebraic expressions inside function applications. An analogous approach using `field_simplify` does not suffice for field expressions, as explained in the following section.

3 The limits of `field_simplify`

Consider

$$\frac{\pi}{\pi^2 + \pi^2} = \frac{1}{2\pi}.$$

`field` can prove this if it proves $\pi^2 + \pi^2 \neq 0$ and $\pi \neq 0$. In practice, `field` reduces the equality to these non-zero side conditions; they still have to be proven separately, for example using the positivity of π. Now consider the variant

$$\log\left(\frac{\pi}{\pi^2 + \pi^2}\right) = \log\left(\frac{1}{2\pi}\right).$$

Here `field` fails because $x \mapsto \log(x)$ is not a field expression. The same issue occurs for any outer function that is not handled by the tactic, such as exp, sin, or cos.

The standard `field` tactic represents expressions as fractions, but it compares those fractions by cross multiplication:

$$q \neq 0 \ \wedge \ s \neq 0 \quad \Longrightarrow \quad \left(\frac{p}{q} = \frac{r}{s} \quad \Longleftrightarrow \quad p \times s = q \times r\right).$$

This means a fraction does not need to be reduced in order to prove an equality. For example, $3\pi/6 = 2\pi/4$ is established by checking the cross-product identity $3\pi \times 4 = 6 \times 2\pi$ together with the non-zero conditions on the denominators. The difficulty is subtler when the fraction occurs inside a function. The tactic can prove that

$$\frac{3\pi}{6} = \frac{2\pi}{4}$$

by cross multiplication, but this proof does not require either fraction to be rewritten to a common representative. In particular, the two sides can remain syntactically different after the `field_simplify` calls.

This distinction does not matter when the equality of the two fractions is the final goal. Once the cross-product equality and the required non-zero conditions have been established, the proof is complete. It matters when the fraction occurs as an argument of a function that is unrecognized by the field normalizer. Consider

$$\cos\left(\frac{3\pi}{6}\right) = \cos\left(\frac{2\pi}{4}\right).$$

The field normalizer can reason about the two fractions, but it does not normalize the surrounding occurrences of `cos`. The two function applications can therefore contain different terms even when their arguments could be proved equal.

The required operation is consequently stronger than proving equality of the two fractions. We need to transform both fractions to a common representative that can be used in the surrounding expression. For the example above, a suitable representative is $\frac{\pi}{2}$.

The desired transformation is therefore

$$\cos\left(\frac{3\pi}{6}\right) \longrightarrow \cos\left(\frac{\pi}{2}\right) \longleftarrow \cos\left(\frac{2\pi}{4}\right).$$

A polynomial gcd provides the missing operation. Given a numerator N and denominator D, we compute a common factor γ and the corresponding quotients ν and δ. This yields a reduced representation of the fraction that can be propagated through an enclosing function application.

4 Making room for an oracle

The existing theorem that justifies the `field_simplify` tactic has the following shape:

$$\mathrm{Fnorm}\, e = r \Longrightarrow \mathrm{FEeval}(l, e) = \frac{\mathrm{Pphi_pow}(l, \mathrm{norm}(\mathrm{num}(r)))}{\mathrm{Pphi_pow}(l, \mathrm{norm}(\mathrm{den}(r)))}.$$

Starting from an arbitrary expression in the studied field, for instance $\frac{1}{\pi} + \frac{1}{cos(1)}$, the reification produces the expression $e = \frac{1}{X_0} + \frac{1}{X_1}$ and the list of bindings $[\pi; cos(1)]$, so that $FEeval(l, e)$ computes to the initial expression $\frac{1}{\pi} + \frac{1}{cos(1)}$. The function Fnorm is a gallina-language function that computes a fraction of ring expressions, together with a collection of expressions that should all be non zero. The name Fnorm is poorly chosen, because this function does not compute a normal form, it only ensures that the expression now has the shape of a single fraction whose numerator and denominator are both ring expressions expressions (of the kind to be processed by the ring machinery). The ring expressions are themselves not guaranteed to be in normal form.

In the case of the input expression $e = \frac{1}{X_0} + \frac{1}{X_1}$, r has the shape $r = \frac{X_1+X_0}{X_0X_1}$, together with the conditions that π and $cos(1)$ should not be 0.

The next step of processing the formula is to compute the ring normal forms of both parts of the fraction, this is performed by the calls to the *norm* function that appear in the right-hand side of the conclusion equality.

The shape of this theorem states clearly that the numerator and the denominator are processed independently. If there is a common factor, it is not removed by the calls to norm.

In our extension to the `field_simplify` tactic, we instead rely on a theorem with the following shape:

$$
\begin{array}{l}
\mathrm{Fnorm}\, e = r \Longrightarrow \\
\forall m\ \nu\ \delta\ \gamma, \\
m \neq 0 \\
m \times \mathrm{norm}(\mathrm{num}(r)) = \nu\gamma \Longrightarrow \\
m \times \mathrm{norm}(\mathrm{denum}(r)) = \delta\gamma \Longrightarrow \\
\mathrm{FEeval}(l, e) = \dfrac{\mathrm{Pphi_pow}(l, \nu)}{\mathrm{Pphi_pow}(l, \delta)}.
\end{array}
$$

In what follows, we name N and D the numerator $\mathrm{norm}(\mathrm{num}(r))$ and the denominator $\mathrm{norm}(\mathrm{denum}(r))$, respectively. In this theorem, m is an integer, and ν, δ, and γ are polynomials in normal form. This theorem is a direct consequence of the previous one, but it makes so that the ultimately produced fraction can be cleaned of common polynomial factors, if these factors are included in the γ polynomial.

The four values m, ν, δ, and γ have to be produced for this theorem to be applied, and this is where the an external computation, which we will call an oracle in the rest of this paper, is needed.

In practice, we rely on an implementation of a Gröbner basis algorithm to compute the greatest common divisor of the two polynomials. This algorithm operates on polynomials with rational coefficient. It is easy to translate our polynomials N D which have integer coefficient, into polynomials with rational coefficients, but when processing the result of the greatest common divisor computation, we need to recover polynomials with integer coefficients. This is where the integer m is needed if we have candidate polynomials ν', δ', γ', we only need to find two numbers $_1$ and m_2, so that $m_1 \times \nu' = \nu$ has integer coefficients, and so do $m_1 \times \delta' = \delta$ and $m_2 \times \gamma' = \gamma$, in this case $m = m_1 m_2$.

4.1 A complete example

The following example illustrates a computation where we take advantage of the gcd computation. Consider

$$\frac{4X^2 - 1}{2X^2 + 3X + 2},$$

The numerator and denominator are

$$N = 4X^2 - 1, \qquad D = 2X^2 + 3X + 2.$$

The polynomial gcd computation returns

$$m = 8, \qquad \gamma = 16X + 8, \qquad \nu = 2X - 1, \qquad \delta = X + 2.$$

These value satisfy the following equalities

$$mN = \nu\gamma, \qquad mD = \delta\gamma. \tag{1}$$

In this example the scale factor is $m = 8$.

The fraction can consequently be represented as

$$\frac{N}{D} = \frac{\nu}{\delta} = \frac{X - 1}{X + 1}.$$

The gcd computation provides a reduced representation of the fraction that can be used to rewrite the argument of f. In particular, the original function application can be rewritten as

$$f\left(\frac{X^2-1}{X^2+2X+1}\right) \longrightarrow f\left(\frac{2X-1}{X+2}\right).$$

The rewrite is justified by the factorization identities **??** verified by Rocq.

4.2 Implementation of the oracle

The Oracle is implemented as an OCaml program, which operates on sequences of monomials to represent multivariate polynomials with rational coefficients. The data type used in the program has the following shape:

```
1 type monomial = { coef: rat; exps: (int * int) list }
2 type mpoly = monomial list
```

A monomial consists of a rational coefficient and a sorted list of pairs (i, e), where i is a variable index and e is its exponent:

For example, the monomial

$$3X_1^2X_4$$

is represented by the coefficient 3 and the exponent list

$$[(1, 2); (4, 1)].$$

Polynomial arithmetic, monomial ordering, reduction, and Buchberger's algorithm operate on this sparse representation. The result is converted back to `poly` when it is returned by the oracle.

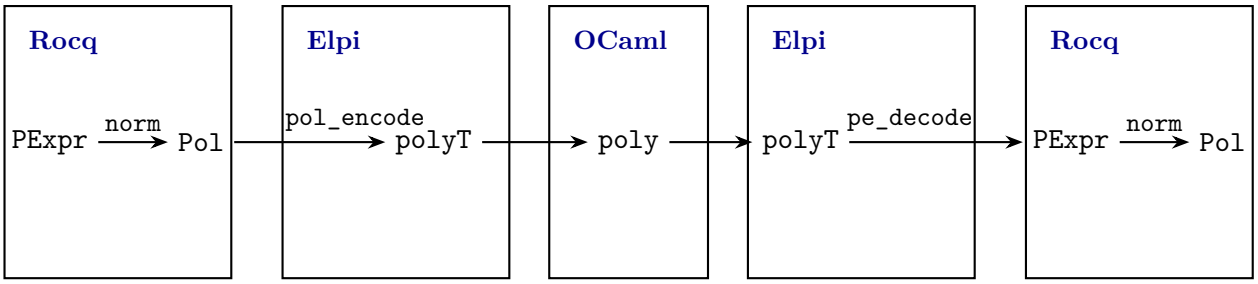
To facilitate the communication with Rocq's implementation of polynomial, we chose to implement an OCaml datatype of ring expressions with integer constants based only on integer constants, variables, addition and multiplication. The datatype description has the following shape:

```
1 type poly =
2 | Const of int | Var of int | Add of poly * poly | Mul of poly * poly
```

This datatype is isomorphic to a subset of the datatype `PExpr` used in the implementation of the ring tactic. Before performing polynomial computations, the implementation converts a `poly` expression to a sparse multivariate representation.

Rational coefficients are used because the polynomial computations can introduce non-integral coefficients. Monomials are ordered lexicographically, with lower variable indices taking precedence. This order determines the leading monomial used in polynomial reduction and Buchberger's algorithm.

We use Buchberger's algorithm as part of the external polynomial computation. This algorithm is well-known and an implementation of it was presented by Théry [**?**]. Developing a new Gröbner basis algorithm is not the objective of this work. Instead, the polynomial computation is treated as an untrusted oracle: its output is used to propose a factorization, and Rocq independently verifies the algebraic identities required to justify the resulting reduction. The correctness of the extension therefore does not depend on trusting the OCaml implementation.



5 The Rocq–ELPI–OCaml interface

5.1 Experimental setup

To work at an early experimental stage, we chose a design that re-uses as much as possible the current implementation of the `ring`-like tactics but avoids modifying any part of the Rocq main program. As a result, our experiment currently works with an unmodified version of the `rocq-core` package, but a modified version of the `rocq-stdlib` package.

The code in `rocq-core` assumes that all behavior of the `ring`-like tactics is governed by 4 lemmas, each of which is an instantiation of a generic lemma parameterized by arbitrary ring or field structures. In our case, we need to use the extra lemma presented in Section **??** that accounts for the possibility of calling an oracle after the reification step. We added such a lemma in the `rocq-stdlib` package, under the name `Field_rw_pow_correct_w_gcd`, its statement is a modification of the existing `Field_rw_pow_correct`, which is used by the existing `field_simplify` tactic.

We then created tactics to harness the extra lemma and the oracle. Ultimately all these tactics should belong in the `rocq-stdlib` package, but since recompiling the `stdlib` package takes about 50 seconds on our computers, we chose to only include in `rocq-stdlib` the most stable part of this harness and to keep the part that was most unstable in a separate package. As a result, we only have to recompile `stdlib` on rare occasions, and compiling the separate package between modifications and tests only takes a fraction of a second.

We call the tactic that is in a separate package the finishing tactic. This tactic is passed as argument to all the new tactics we added to the `rocq-stdlib` package under the name `finish_tac`.

Four of the new tactics are variations of `field_simplify`, `Field_simplify`, `Field_simplify_gen`, `Field_norm_gen`, and they are named by adding the `_gcd` suffix to the name. Two of the new tactics are variations of `ApplyLemmaThen`, `ApplyLemmaThenAndCont` which are drawn from the `ring_simplify` tactic implementation, but we preferred renaming them as `rewrites` and `one_of_rewrites`.

In practice, the only change implemented by these variations is to make it possible to choose the lemma that is being used in the proof, and to call the finishing tactic for each of the rewrites performed by `one_of_rewrites`.

Our current implementation can be adapted statically to any field structure, and we specialized it to the field structure of the type `R` of real numbers. The usual behavior of `field_simplify` which adapt dynamically to the field structure determined by the type of formulas found as argument is not available, as this would require modifying the `rocq-core` package.

5.2 Polynomial representations

We need data representing polynomials to be transferred from Rocq to the OCaml (the oracle's implementation language) and back. Writing directly a converter between these two worlds is difficult, and we preferred to use some ELPI capabilities as a broker.

We defined in `Elpi` a datatype called `polyT` which is an exact mirror of the datatype `poly` that we use in the OCaml implementation of the oracle. Elpi also provides an easy way to write foreign function calls, so that we have an easy way to transfer data of type `polyT` to data of type `poly` in the oracle and call the main function of the oracle on this data. This foreign function interface is described in more detail in Section **??**.

Rocq uses two types to represent ring expressions: `PExpr` where the constructors describe directly the operation of addition, multiplication, and so on, and `Pol` where constructors describe the structure of polynomials in normal form. The function `norm` mentioned in Section **??** actually produces objects of type `Pol`.

The ELPI language provides powerful inspection mechanisms and interfaces to Rocq, so that it easy to write function converting Rocq data into ELPI Data. In the present case, we wrote an ELPI function called `pol_encode` converting polynomial of type `Pol` (in Rocq) to values of type `polyT` (in ELPI).

For example, the polynomial

$$X_1 + X_1^2$$

can be represented in `PExpr` as

```
PEadd (PEX 1) (PEmul (PEX 1) (PEX 1)).
```

The function `norm` converts this expression into its canonical `Pol` representation, passing this value to `pol_encode` then produces the following result encoded as

```
add (var 1) (mul (var 1) (var 1)).
```

5.3 ELPI as the bridge

ELPI is a Prolog-like language for term manipulation [**?**]. The integration of ELPI with Rocq builds on work on implementing type theory in higher-order constraint logic programming [**?**]. Its foreign-function interface is implemented in OCaml and connects ELPI to OCaml code loaded by the Rocq plugin.

To establish the correspondence between type `polyT` (in ELPI) and type `poly` (in OCAML) we only need to write some systematic code that makes it explicit that the `add` symbol in ELPI is connected to the `Add` constructor in OCaml, the type of both its arguments are the same as the type produced by `add`, and pattern matching on a term with the `Add` constructor is the same as inspecting the subterms of a `add` term. This correspondance is expressed by the following piece of code (looking only at the description for `add`).

```
1 K("add", "binary␣addition", S(S(N)),
2   B (fun x y -> Add (x, y)),
3   M (fun ~ok ~ko t ->
4     match t with Add (x, y) -> ok x y | _ -> ko ()))
```

Here the `K` expresses that we are describing a constructor correspondance, The text `S (S (N))` expresses that `add` has two arguments, whose is in the type currently being described. Line 2 shows how to build a term in OCaml corresponding to a term in ELPI whose head symbol is `add`. This

line is especially useful when translating terms from ELPI to OCaml. Conversely, lines 3 and 4 describe how to convert data from OCaml to ELPI.

This description for `add` is combined with descriptions for `mul`, `var` (for variables), and `pcst` (for constants) in a declaration whose processing generates at the same time type declarations to be loaded in ELPI and OCaml code that will perform the conversions when needed.

Similarly, the oracle's main function must be exposed as a ELPI function. In OCaml, this function is provided with type:

```
Gcd.poly_gcd :  poly -> poly -> (int * poly * poly * poly).
```

The foreign function interface lets us declare a predicate `gcd_poly` in ELPI, where different parts of the result appear as different arguments of the predicate.

When processing the declaration of `polyT` and `gcd_poly`, the foreign function tool produces the following declarations to be loaded in ELPI.

```
1 external symbol pcst : int -> polyT = "1".  % constant polynomial
2 external symbol var : int -> polyT = "1".  % indexed variable
3 external symbol add : polyT -> polyT -> polyT = "1".  % binary addition
4 external symbol mul : polyT -> polyT -> polyT = "1".  % binary multiplication
5
6 % [gcd_poly p1 p2 factor gcd p1_div p2_div] gcd is the greatest common
7 % divisor of polynomials p1 and p2,up to an integer factor
8 %  p1_div is (factor * p1)/gcd, p2_div is (factor * p2)/gcd
9 external func gcd_poly polyT, polyT -> int, polyT, polyT, polyT.
```

It also produces some OCaml code that can be loaded in Rocq at the same time as the oracle implementation. With all this, any call to the ELPI function `gcd_poly` with two inputs of type `polyT` triggers a call to `Gcd.poly_gcd` in OCaml, with the two corresponding inputs of type `poly`. The results of this execution are then converted back to type `polyT`.

In this code fragment, % marks the beginning of a comment.

The main justification theorem requires the various polynomials to be in type `Pol`, but the ELPI function `gcd_poly` returns values in type `polyT`. The easiest way to reconcile the need is to translate each term of type `polyT` into the corresponding term of type `PExpr`. This easy because the type `PolyT` is isomorphic to a sub-language of `PExpr`. The value of type `Pol` can then be obtained by calling again the `norm` function.

the function `gcd_poly` has inputs and outputs in types `polyT` and `int`. In the end we wrap this function into a ELPI function that takes as inputs terms of the Rocq language, representing these polynomials and numbers. This is done with the following fragments of code:

```
1  func pol_decode polyT -> term.
2  pol_decode PolyT_a Result :-
3    pe_decode PolyT_a Pe_a,
4    coq.reduction.vm.norm {{norm lp:Pe_a}} {{Pol Z}} Result.
5
6  func gcd_and_factors term, term -> term, term, term, term.
7  gcd_and_factors N D Ne De Gcd MZ :-
8    pol_encode N Nx, pol_encode D Dx,
9    gcd_poly Nx Dx M G N' D',
10   pol_decode N' Ne, pol_decode D' De,
11   pol_decode G Gcd, z_decode M MZ.
```

In turn, the computation can be made available to the language in which the `ring`-like tactics have been programmed, namely the `LTac` language.

```
Elpi Tactic factorize_by_gcd.
Elpi Accumulate Plugin "ext.elpi".
Elpi Accumulate File encode.
Elpi Accumulate lp:{{

solve (goal _ _ _ _ [trm N, trm D] as G) GL :-
  gcd_and_factors N D N' D' Gcd LCM,
  (refine {{(lp:LCM, (lp:N', (lp:D', lp:Gcd)))}} G GL).
}}.
```

This describes how a piece of ELPI code can be viewed in Ltac as a Ltac function that returns a value (in this case a Rocq quadruplet). To call this function, it will be necessary to write `Elpi factorize_by_gcd N D`.

5.4 Combining the oracle with the extended theorem

We can now come back closer to the Rocq prover, where we only need to finish programming our Ltac tactic that uses the result of the oracle in combination with the extended theorem `Field_rw_pow_correct_w_gcd` (also described in Section **??**). The proof script still lives in Ltac, but the actual computation is delegated to ELPI. The bridge is explicit: an Ltac function builds a Rocq term, calls the ELPI computation, and receives back the scale factor, reduced numerator, reduced denominator, and gcd.

For each of the formulas that we want to process with `field_simplify`, the formula has already been reified, the numerator and denominator have already been normalized (as ring formulas), and the theorem has already been instantiated with this reified formula. The tactic `fraction_finisher` is then called with the initial formula (in variable `Term`), the instantiated theorem (in variable `hyp`), the denominator (in variable `D`) and the numerator (in variable `N`). The tactic performs the following actions:

```
1  Ltac fraction_finisher Term hyp D N :=
2  let hyp2 := fresh "rew_l2" in
3  let res :=
4    constr:(ltac:(elpi factorize_by_gcd ltac_term:(N) ltac_term:(D))) in
5  let F := eval cbv [fst] in (fst res) in
6  let N2 := eval cbv [fst snd] in (fst (snd res)) in
7  let D2 := eval cbv [fst snd] in (fst (snd (snd res))) in
8  let Gcd := eval cbv [snd fst] in (snd (snd (snd res))) in
9  assert (hyp2 := hyp F N2 D2 Gcd);
10 lazymatch type of hyp2 with
11 | _ -> _ -> _ -> _ -> ?t = ?r =>
12 change t with Term in hyp2;
13   (try rewrite hyp2; clear hyp hyp2);
14   [reduce_Pphi_pow | easy| easy | easy| reduce_PCond]
15 end.
```

At line 4, the oracle is being called and the result stored in variable `res`. In lines 5 to 8 the various parts are retrieved, and at line 9, the theorem is further instantiated with these parts.

After this instantiation the theorem statement has the shape of an equality that is valid solely under 4 assumptions (this is checked on lines 10-11). the first assumption is that the number m is non zero. The next two assumptions are the factorization identities, the last assumption is the collection of non-zero conditions that was collected at the time of reification.

On ligne 12, we change the left-hand side of the equality with the initial formula, because it is convertible when reification is correct, then we rewrite with this conditional equality, thus obtaining 5 goals. The first one is the where we performed the rewriting operation, in this goal all polynomial normalization operations should be performed (tactic `reduce_Pphi_pow` on line 40). The next three goals should be trivially provable if the oracle performed its job correctly, hence the three calls to the tactic `easy` on line 14. We have no way to make sure that the last goal can be proved automatically, but the formula should should be cleaned up so all functions coming from the `field` infrastructure are reduced (tactic `reduce_Pcond` on line 40).

5.5 A few examples

We defined a new tactic named `field_simplify'` that integrates this gcd computation.

Given a goal statement of the following form,

```
1 cos (2 * PI / 4) = cos (3 * PI / 6)
```

Tactic `field_simplify (2 * PI / 4)` does not change the goal. On the other hand, `field_simplify' (2 * PI / 4)` does replace `(2 * PI / 4)` with `(PI / 2)`.

6 Integrating with `field`

The gcd computation is integrated into the existing `field` simplification procedure rather than implemented as a separate user-facing tactic. The extension replaces the part of the simplification process that normally handles the numerator and denominator with a parameterizable Ltac function.

In a typical call, the function receives the term being rewritten, the hypothesis produced by field normalization, and the denominator and numerator:

```
1 fraction_finisher Term hyp D N.
```

It then calls `factorize_by_gcd` to obtain the scale factor, reduced numerator, reduced denominator, and gcd. The tactic uses `easy` to discharge the non-zero side condition when the available hypotheses suffice. It then applies the existing field-simplification theorem using the reduced numerator and denominator.

The integration is therefore deliberately asymmetric: the existing `field` infrastructure remains responsible for identifying and simplifying field expressions, while the new gcd computation supplies the common representation needed when the resulting fraction occurs inside a function outside the field fragment. The ELPI entry point is consequently an implementation subpart of the extended simplification procedure rather than a new user-facing tactic.

7 Quantitative evaluation

To evaluate the computational cost of the approach, we consider a family of increasingly nested expressions defined by the following recursive functions:

```
1 Fixpoint r (n : nat) (x : R) : R :=
2   match n with
3   | O => x
4   | S n' => sin ((2*x - 2)* r n' x /2)
5   end.
```

```
6
7  Fixpoint r_alt (n : nat) (x : R) : R :=
8    match n with
9    | O => x
10   | S n' => sin ((x^2 - 1) * r_alt n' x / (x + 1))
11   end.
```

For each depth n, we prove the following equality

$$forall x : R, forall p : x + 1 <> 0, r(n, x) = r_{\text{alt}}(n, x).$$

The proof unfolds both recursive definitions, applies `deep_field`, and finishes with `reflexivity` and `nra` to get rid of side conditions. The experiment therefore measures the cost of simplifying algebraic expressions nested inside successive applications of sin.

The benchmark measures both the tactic execution time and the total wall-clock time. The former is obtained using Rocq's `Time` command, whereas the latter includes the overhead of processing and compiling the surrounding proof.

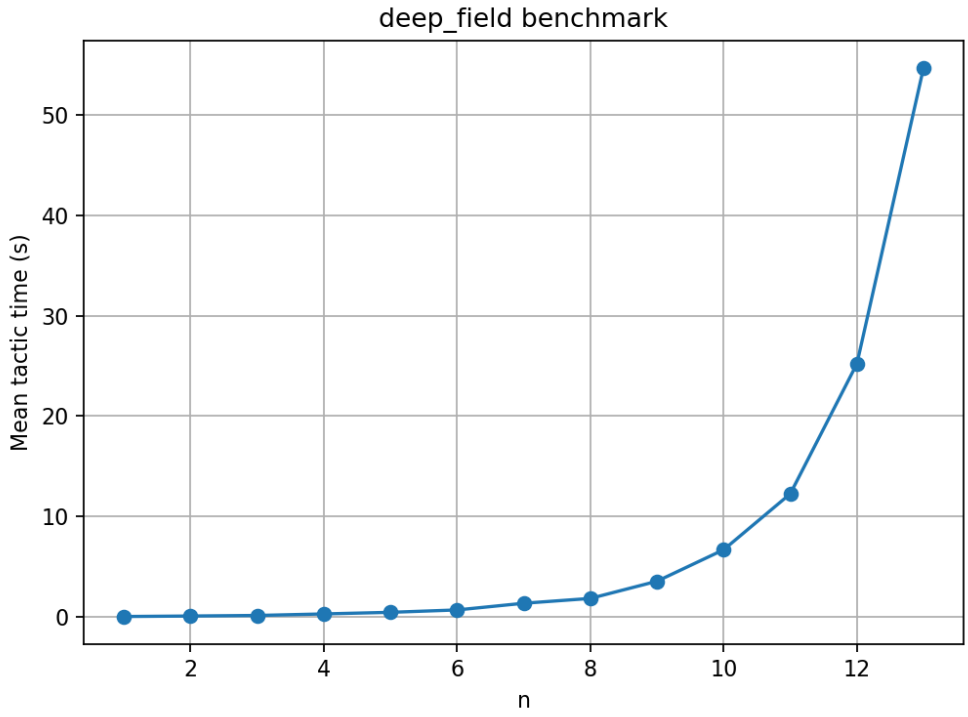


Figure 1: Measured execution times as a function of nesting depth n.

We evaluated 13 instances, for $(1 \leq n \leq 13)$, running each instance 10 times and reporting the mean execution time. The mean tactic execution time increased from (0.038) seconds at (n=1) to (54.715) seconds at (n=13), while the mean wall-clock time increased from (0.499) seconds to (74.090) seconds.

The execution time increases rapidly with the nesting depth. However, the practical relevance of this growth depends on the depths encountered in actual proofs. For undergraduate mathematical students it is unlikely to encounter more than say depth 5 formula, so the cost is acceptable.

8 Lessons learned

The implementation separates polynomial computation from proof construction. The OCaml oracle proposes polynomial factors, while Rocq verifies the algebraic identities needed to justify their use. This separation allows the extension to benefit from external computation without trusting the OCaml implementation for correctness.

The experiment also highlights an important difference between deep ring and deep field simplification. For ring expressions, normalization naturally provides a canonical polynomial representation, so recursively simplifying function arguments is sufficient to make equivalent terms syntactically equal. For field expressions, equality can instead be established by cross multiplication without producing such a common representation. The gcd computation addresses this additional representation problem by making it possible to reduce the fraction.

The current extension nevertheless operates at a limited point in the field simplification process. It handles the field expression selected by the existing field-simplification infrastructure, but it does not yet recursively traverse arbitrary nested field expressions in the same way as the deep ring procedure. A fully general deep field simplification procedure would therefore require extending the recursive traversal.

9 Conclusion

Extending reflexive algebraic tactics to expressions occurring inside unrecognized function applications requires more than recursively invoking the existing normalization procedures. For ring expressions, polynomial normalization already provides a canonical representation that can be propagated through the surrounding function application. For field expressions, cross multiplication can prove equality without producing such a representation, so an additional reduction step is required.

This work uses a polynomial gcd computation to provide that reduction. An OCaml oracle computes the common factor and the corresponding numerator and denominator quotients, while Rocq verifies the resulting factorization identities and side conditions. ELPI provides the interface between the external computation and the existing Ltac and field-simplification infrastructure.

A natural next step is to extend the approach to a general deep field-simplification procedure that processes nested expressions from the inside out. This could also reduce unnecessary computation and the number of generated side conditions, but its effectiveness must be evaluated experimentally. Measurements of proof success, execution time, and generated side conditions would help assess the practical benefits of the approach.

The broader objective is to make algebraic automation behave more naturally on mathematical expressions, while preserving the proof guarantees of Rocq. The combination of recursive simplification, polynomial factorization, and proof-producing normalization provides a foundation for pursuing this objective beyond the ring and field fragments.

References

[1] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λprolog interpreter. In Martin Davis, Ansgar Fehnker, Annabelle McIver, and Andrei Voronkov, editors, *Logic for Programming, Artificial Intelligence, and Reasoning - 20th International Conference, LPAR-20 2015, Suva, Fiji, November 24-28, 2015, Proceedings*, volume 9450 of *Lecture Notes in Computer Science*, pages 460–468. Springer, 2015.

[2] Benjamin Grégoire and Assia Mahboubi. Proving equalities in a commutative ring done right in coq. In *Theorem Proving in Higher Order Logics*, pages 98–113. Springer, 2005.

[3] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Math. Struct. Comput. Sci.*, 29(8):1125–1150, 2019.

[4] The Rocq Development Team. The rocq prover. `https://doi.org/10.5281/zenodo.17473943`, 2025. Version 9.1.

[5] Laurent Théry. A machine-checked implementation of buchberger's algorithm. *Journal of Automated Reasoning*, 26(2):107–137, 2001.